

C PROGRAM CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#define MAX_PACKAGES 100

#define MAX_CAPACITY 500

typedef struct

{

    int id;

    int weight;

    int volume;

    int value;

    int priority;

} Package;

/* Priority text */

void printPriority(int p)

{

    if (p == 3)

        printf("High");

    else if (p == 2)

        printf("Medium");

    else

        printf("Low");

}

/* Display selected packages */

void displayResult(char name[], Package p[], int selected[],

    int n, int weightCap, int volumeCap)
```

```

{
    int i;
    int totalWeight = 0;
    int totalVolume = 0;
    int totalValue = 0;
    int count = 0;

    printf("\n=====\\n");
    printf("    %s RESULT\\n", name);
    printf("=====\\n");
    printf("\\nSelected Packages:\\n");
    for (i = 0; i < n; i++)
    {
        if (selected[i] == 1)
        {
            printf("P%d Weight=%d kg Volume=%d L Value=Rs.%d Priority=",
                p[i].id,
                p[i].weight,
                p[i].volume,
                p[i].value);

            printPriority(p[i].priority);
            printf("\\n");

            totalWeight += p[i].weight;
            totalVolume += p[i].volume;
            totalValue += p[i].value;
            count++;
        }
    }
}

```

```

printf("\nTotal Packages   : %d", count);
printf("\nTotal Weight    : %d kg", totalWeight);
printf("\nTotal Volume     : %d L", totalVolume);
printf("\nTotal Value      : Rs.%d", totalValue);
printf("\nWeight Utilization : %.2f%%",
       (totalWeight * 100.0) / weightCap);
printf("\nVolume Utilization : %.2f%%",
       (totalVolume * 100.0) / volumeCap);
printf("\n");
}
/* =====

```

DYNAMIC PROGRAMMING

```
===== */
```

```

void dynamicProgramming(Package p[], int n,
                        int capacity, int selected[])
{
    int dp[MAX_PACKAGES + 1][MAX_CAPACITY + 1];
    int i, w;
    for (i = 0; i <= n; i++)
    {
        for (w = 0; w <= capacity; w++)
        {
            dp[i][w] = 0;
        }
    }
    for (i = 1; i <= n; i++)

```

```

{
    for (w = 0; w <= capacity; w++)
    {
        if (p[i - 1].weight <= w)
        {
            int include =
                p[i - 1].value +
                dp[i - 1][w - p[i - 1].weight];
            int exclude =
                dp[i - 1][w];

            if (include > exclude)
                dp[i][w] = include;
            else
                dp[i][w] = exclude;
        }
        else
        {
            dp[i][w] = dp[i - 1][w];
        }
    }
}

/* Backtracking */
w = capacity;
for (i = n; i > 0; i--)
{
    if (dp[i][w] != dp[i - 1][w])

```

```

        {
            selected[i - 1] = 1;
            w -= p[i - 1].weight;
        }
    }
}

/* =====

SORT FOR GREEDY

===== */

void sortPackages(Package p[], int n)
{
    int i, j;
    Package temp;
    double score1, score2;
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            score1 =
                (double)(p[i].value * p[i].priority)
                / p[i].weight;
            score2 =
                (double)(p[j].value * p[j].priority)
                / p[j].weight;
            if (score2 > score1)
            {
                temp = p[i];

```

```

        p[i] = p[j];
        p[j] = temp;
    }
}
}
}
}
/* =====

GREEDY
===== */

void greedy(Package original[], int n,
            int weightCap, int volumeCap,
            int selected[])
{
    Package p[MAX_PACKAGES];
    int i;
    int weight = 0;
    int volume = 0;
    for (i = 0; i < n; i++)
        p[i] = original[i];
    sortPackages(p, n);
    for (i = 0; i < n; i++)
    {
        if (weight + p[i].weight <= weightCap &&
            volume + p[i].volume <= volumeCap)
        {
            selected[p[i].id - 1] = 1;

```

```

        weight += p[i].weight;
        volume += p[i].volume;
    }
}
}
/* =====

```

HYBRID

Greedy + Local Improvement

```

===== */

void hybrid(Package p[], int n,
            int weightCap, int volumeCap,
            int selected[])
{
    int i, j;
    int weight = 0;
    int volume = 0;
    int value = 0;
    double score[MAX_PACKAGES];
    /* Calculate priority-aware score */
    for (i = 0; i < n; i++)
    {
        score[i] =
            (double)(p[i].value * p[i].priority)
            / (p[i].weight + p[i].volume * 0.5);
    }
    /* Greedy initial solution */
    for (i = 0; i < n - 1; i++)

```

```

{
    for (j = i + 1; j < n; j++)
    {
        if (score[j] > score[i])
        {
            double temp = score[i];
            score[i] = score[j];
            score[j] = temp;
            Package tempP = p[i];
            p[i] = p[j];
            p[j] = tempP;
        }
    }
}

for (i = 0; i < n; i++)
{
    if (weight + p[i].weight <= weightCap &&
        volume + p[i].volume <= volumeCap)
    {
        selected[p[i].id - 1] = 1;
        weight += p[i].weight;
        volume += p[i].volume;
        value += p[i].value;
    }
}

/* Local improvement */

for (i = 0; i < n; i++)

```



```

{
    if (selected[i] == 0)
    {
        for (j = 0; j < n; j++)
        {
            if (selected[j] == 1)
            {
                int newWeight =
                    weight -
                    p[j].weight +
                    p[i].weight;
                int newVolume =
                    volume -
                    p[j].volume +
                    p[i].volume;
                int newValue =
                    value -
                    p[j].value +
                    p[i].value;
                if (newWeight <= weightCap &&
                    newVolume <= volumeCap &&
                    newValue > value)
                {
                    selected[j] = 0;
                    selected[i] = 1;
                    weight = newWeight;
                    volume = newVolume;

```



```

int weightCapacity;
int volumeCapacity;
int dpSelected[MAX_PACKAGES] = {0};
int greedySelected[MAX_PACKAGES] = {0};
int hybridSelected[MAX_PACKAGES] = {0};
clock_t start, end;
double dpTime;
double greedyTime;
double hybridTime;
printf("\n");
printf("=====\\n");
printf("    LOGISTICS RESOURCE ALLOCATION SYSTEM\\n");
printf("=====\\n");
printf("\nNumber of Packages : %d\\n", n);
printf("\nEnter Vehicle Weight Capacity (kg): ");
scanf("%d", &weightCapacity);
printf("Enter Vehicle Volume Capacity (L): ");
scanf("%d", &volumeCapacity);
/* Package information */
printf("\n=====\\n");
printf("    PACKAGE DATA\\n");
printf("=====\\n");
printf("\nID\tWeight\tVolume\tValue\tPriority\\n");
int i;
for (i = 0; i < n; i++)
{

```

```

        printf("P%d\t%d\t%d\tRs.%d\t",
               packages[i].id,
               packages[i].weight,
               packages[i].volume,
               packages[i].value);
        printPriority(packages[i].priority);
        printf("\n");
    }
    /* ===== DP ===== */
    start = clock();
    dynamicProgramming(
        packages,
        n,
        weightCapacity,
        dpSelected
    );
    end = clock();
    dpTime =
        ((double)(end - start) / CLOCKS_PER_SEC)
        * 1000;
    /* ===== GREEDY ===== */
    start = clock();
    greedy(
        packages,
        n,
        weightCapacity,
        volumeCapacity,

```

```

        greedySelected
    );
    end = clock();
    greedyTime =
        ((double)(end - start) / CLOCKS_PER_SEC)
        * 1000;

/* ===== HYBRID ===== */
    start = clock();
    hybrid(
        packages,
        n,
        weightCapacity,
        volumeCapacity,
        hybridSelected
    );
    end = clock();

    hybridTime =
        ((double)(end - start) / CLOCKS_PER_SEC)
        * 1000;

/* ===== RESULTS ===== */
    displayResult(
        "DYNAMIC PROGRAMMING",
        packages,
        dpSelected,
        n,

```

```
        weightCapacity,  
        volumeCapacity  
    );  
  
    printf("\nExecution Time: %.4f ms\n", dpTime);  
  
    displayResult(  
        "GREEDY",  
        packages,  
        greedySelected,  
        n,  
        weightCapacity,  
        volumeCapacity  
    );  
    printf("\nExecution Time: %.4f ms\n", greedyTime);  
    displayResult(  
        "HYBRID",  
        packages,  
        hybridSelected,  
        n,  
        weightCapacity,  
        volumeCapacity  
    );  
  
    printf("\nExecution Time: %.4f ms\n", hybridTime);
```

```

/* ===== COMPARISON ===== */

printf("\n\n=====\\n");
printf("      ALGORITHM COMPARISON\\n");
printf("=====\\n");

printf("\nAlgorithm\t\tExecution Time\\n");
printf("-----\\n");

printf("Dynamic Programming\t%.4f ms\\n", dpTime);
printf("Greedy\t\t%.4f ms\\n", greedyTime);
printf("Hybrid\t\t%.4f ms\\n", hybridTime);
printf("\n=====\\n");
printf("      CONCLUSION\\n");
printf("=====\\n");
printf("\nDynamic Programming provides an optimal solution\\n");
printf("for the weight-based knapsack formulation.\\n");
printf("\nGreedy provides a fast approximate solution using\\n");
printf("value and priority based package selection.\\n");
printf("\nHybrid combines greedy selection with local\\n");
printf("improvement to obtain a better practical solution.\\n");
printf("\nThe Hybrid approach is suitable for large-scale\\n");
printf("logistics resource allocation because it balances\\n");
printf("solution quality and computational efficiency.\\n");
printf("\n=====\\n")
return 0;
}

```

HTML IMPLEMENTATION CODE:

```
```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>LogiOpt - Resource Allocation</title>

<style>

/* =====

GLOBAL
===== */

* {
 margin: 0;
 padding: 0;
 box-sizing: border-box;
}

body {
 font-family: "Segoe UI", Arial, sans-serif;
 background: #f5f7fb;
 color: #172033;
}
```



```
/* =====

 LAYOUT

===== */
```

```
.app {
 display: flex;
 min-height: 100vh;
}
```

```
/* =====

 SIDEBAR

===== */
```

```
.sidebar {
 width: 245px;
 background: #111827;
 color: white;
 padding: 25px 18px;
 position: fixed;
 left: 0;
 top: 0;
 bottom: 0;
}
```

```
.logo {
 display: flex;
```

```
 align-items: center;
 gap: 12px;
 margin-bottom: 40px;
}
```

```
.logo-icon {
 width: 42px;
 height: 42px;
 background: #2563eb;
 border-radius: 12px;
 display: flex;
 align-items: center;
 justify-content: center;
 font-size: 22px;
}
```

```
.logo h2 {
 font-size: 20px;
}
```

```
.logo span {
 color: #60a5fa;
}
```

```
.menu-title {
 color: #9ca3af;
 font-size: 11px;
```

```
text-transform: uppercase;
margin: 20px 10px 10px;
}
```

```
.menu-item {
padding: 13px 14px;
border-radius: 9px;
margin-bottom: 6px;
color: #d1d5db;
cursor: pointer;
}
```

```
.menu-item:hover,
.menu-item.active {
background: #1d4ed8;
color: white;
}
```

```
.sidebar-bottom {
position: absolute;
bottom: 25px;
left: 20px;
right: 20px;
padding: 15px;
border-radius: 10px;
background: #1f2937;
}
```

```
.sidebar-bottom small {
 color: #9ca3af;
}
```

```
/* =====

 MAIN

===== */
```

```
.main {
 margin-left: 245px;
 width: calc(100% - 245px);
}
```

```
/* =====

 TOPBAR

===== */
```

```
.topbar {
 height: 75px;
 background: white;
 border-bottom: 1px solid #e5e7eb;
 display: flex;
 justify-content: space-between;
 align-items: center;
 padding: 0 35px;
}
```

```
.topbar h1 {
 font-size: 22px;
}
```

```
.topbar p {
 color: #6b7280;
 font-size: 13px;
 margin-top: 4px;
}
```

```
.status {
 display: flex;
 align-items: center;
 gap: 8px;
 background: #ecfdf5;
 padding: 9px 14px;
 border-radius: 20px;
 color: #047857;
 font-size: 13px;
}
```

```
.status-dot {
 width: 8px;
 height: 8px;
 border-radius: 50%;
 background: #10b981;
```

```
}
```

```
/* =====
```

```
CONTENT
```

```
===== */
```

```
.content {
```

```
padding: 30px 35px;
```

```
}
```

```
.section-title {
```

```
font-size: 18px;
```

```
margin-bottom: 15px;
```

```
}
```

```
/* =====
```

```
KPI CARDS
```

```
===== */
```

```
.kpi-grid {
```

```
display: grid;
```

```
grid-template-columns: repeat(4, 1fr);
```

```
gap: 18px;
```

```
margin-bottom: 25px;
```

```
}
```

```
.kpi {
```

```
background: white;
padding: 20px;
border-radius: 14px;
border: 1px solid #e5e7eb;
position: relative;
overflow: hidden;
}
```

```
.kpi-icon {
width: 42px;
height: 42px;
background: #fff6ff;
color: #2563eb;
border-radius: 10px;
display: flex;
align-items: center;
justify-content: center;
font-size: 20px;
margin-bottom: 15px;
}
```

```
.kpi h4 {
color: #6b7280;
font-size: 13px;
font-weight: 500;
}
```

```
.kpi h2 {
 font-size: 25px;
 margin-top: 5px;
}
```

```
/* =====

CARDS

===== */
```

```
.card {
 background: white;
 border: 1px solid #e5e7eb;
 border-radius: 14px;
 padding: 22px;
 margin-bottom: 22px;
}
```

```
.card-header {
 display: flex;
 justify-content: space-between;
 align-items: center;
 margin-bottom: 18px;
}
```

```
.card-header h3 {
 font-size: 17px;
}
```



```
.card-header p {
 color: #6b7280;
 font-size: 12px;
}
```

```
/* =====

CONTROLS
===== */
```

```
.control-grid {
 display: grid;
 grid-template-columns: 1fr 1fr;
 gap: 20px;
}
```

```
label {
 display: block;
 font-size: 13px;
 color: #4b5563;
 margin-bottom: 7px;
 font-weight: 600;
}
```

```
input,
select {
 width: 100%;
```

```
padding: 12px;
border: 1px solid #d1d5db;
border-radius: 8px;
outline: none;
background: white;
font-size: 14px;
}
```

```
input:focus,
select:focus {
 border-color: #2563eb;
}
```

```
/* =====
BUTTONS
===== */
```

```
button {
 border: none;
 padding: 11px 17px;
 border-radius: 8px;
 cursor: pointer;
 font-weight: 600;
 font-size: 13px;
 transition: 0.2s;
}
```

```
.primary {
 background: #2563eb;
 color: white;
}
```

```
.primary:hover {
 background: #1d4ed8;
}
```

```
.green {
 background: #059669;
 color: white;
}
```

```
.green:hover {
 background: #047857;
}
```

```
.gray {
 background: #f3f4f6;
 color: #374151;
}
```

```
.gray:hover {
 background: #e5e7eb;
}
```

```
.red {
 background: #fee2e2;
 color: #b91c1c;
}
```

```
.red:hover {
 background: #fecaca;
}
```

```
.button-row {
 display: flex;
 gap: 10px;
 flex-wrap: wrap;
 margin-top: 15px;
}
```

```
/* =====

 ALGORITHM BUTTONS

===== */
```

```
.algorithm-grid {
 display: grid;
 grid-template-columns: repeat(3, 1fr);
 gap: 15px;
}
```

```
.algorithm {
```

```
padding: 20px;
border: 2px solid #e5e7eb;
border-radius: 12px;
background: white;
cursor: pointer;
transition: 0.2s;
}
```

```
.algorithm:hover {
 border-color: #2563eb;
 transform: translateY(-2px);
}
```

```
.algorithm strong {
 display: block;
 font-size: 15px;
 margin-bottom: 6px;
}
```

```
.algorithm small {
 color: #6b7280;
}
```

```
.algorithm-icon {
 font-size: 25px;
 margin-bottom: 12px;
}
```

```
/* =====
```

```
TABLE
```

```
===== */
```

```
.table-container {
 overflow-x: auto;
}
```

```
table {
 width: 100%;
 border-collapse: collapse;
}
```

```
th {
 background: #f8fafc;
 color: #64748b;
 font-size: 12px;
 text-transform: uppercase;
 padding: 13px;
}
```

```
td {
 padding: 13px;
 border-top: 1px solid #edf0f4;
 text-align: center;
 font-size: 13px;
```

```
}
```

```
.priority {
 padding: 5px 10px;
 border-radius: 15px;
 font-size: 11px;
 font-weight: bold;
}
```

```
.high {
 background: #fee2e2;
 color: #dc2626;
}
```

```
.medium {
 background: #fef3c7;
 color: #b45309;
}
```

```
.low {
 background: #dcfce7;
 color: #15803d;
}
```

```
/* =====
```

RESULT GRID

```
===== */
```

```
.result-grid {
 display: grid;
 grid-template-columns: repeat(5, 1fr);
 gap: 12px;
}
```

```
.result {
 padding: 15px;
 background: #f8fafc;
 border-radius: 10px;
 text-align: center;
}
```

```
.result span {
 color: #64748b;
 font-size: 11px;
}
```

```
.result strong {
 display: block;
 font-size: 20px;
 margin-top: 5px;
}
```

```
/* =====
```

PROGRESS



```
===== */
```

```
.progress-section {
 margin-top: 20px;
}
```

```
.progress-label {
 display: flex;
 justify-content: space-between;
 font-size: 12px;
 margin-bottom: 7px;
}
```

```
.progress {
 width: 100%;
 height: 10px;
 background: #e5e7eb;
 border-radius: 10px;
 overflow: hidden;
}
```

```
.progress-bar {
 height: 100%;
 width: 0%;
 background: #2563eb;
 transition: 0.5s;
}
```

```
/* =====
```

## COMPARISON

```
===== */
```

```
.comparison-grid {
 display: grid;
 grid-template-columns: repeat(3, 1fr);
 gap: 18px;
}
```

```
.compare-card {
 padding: 20px;
 background: #f8fafc;
 border-radius: 12px;
}
```

```
.compare-card h4 {
 margin-bottom: 15px;
}
```

```
.metric {
 margin-bottom: 12px;
}
```

```
.metric-header {
 display: flex;
```

```
 justify-content: space-between;
 font-size: 12px;
 margin-bottom: 5px;
}
```

```
/* =====
FOOTER
===== */
```

```
footer {
 text-align: center;
 padding: 25px;
 color: #9ca3af;
 font-size: 12px;
}
```

```
/* =====
RESPONSIVE
===== */
```

```
@media(max-width:1000px) {
```

```
 .sidebar {
 width: 200px;
 }
```

```
 .main {
```

```
margin-left: 200px;
width: calc(100% - 200px);
}
```

```
.kpi-grid {
 grid-template-columns: repeat(2, 1fr);
}
```

```
.algorithm-grid,
.comparison-grid {
 grid-template-columns: 1fr;
}
```

```
.result-grid {
 grid-template-columns: repeat(2, 1fr);
}
}
```

```
@media(max-width:700px) {
```

```
.sidebar {
 display: none;
}
```

```
.main {
 margin-left: 0;
 width: 100%;
```

```
}
```

```
.content {
 padding: 20px;
}
```

```
.topbar {
 padding: 0 20px;
}
```

```
.control-grid {
 grid-template-columns: 1fr;
}
```

```
.kpi-grid {
 grid-template-columns: 1fr;
}
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="app">
```

```
<!-- ===== SIDEBAR ===== -->
```

```
<aside class="sidebar">
```

```
<div class="logo">
```

```
<div class="logo-icon"> 🚚 </div>
```

```
<h2>LogiOpt</h2>
```

```
</div>
```

```
<div class="menu-title">Navigation</div>
```

```
<div class="menu-item active">
```

```
 📊 Dashboard
```

```
</div>
```

```
<div class="menu-item">
```

```
 📦 Packages
```

```
</div>
```

```
<div class="menu-item">
```

```
 🚚 Vehicles
```

```
</div>
```

```
<div class="menu-item">
```

⚙ Optimization

</div>

<div class="menu-title">Analysis</div>

<div class="menu-item">

📊 Performance

</div>

<div class="menu-item">

📄 Reports

</div>

<div class="sidebar-bottom">

<strong>Optimization Engine</strong>

<br>

<small>DP • Greedy • Hybrid</small>

</div>

</aside>

<!-- ===== MAIN ===== -->

```
<main class="main">
```

```
<!-- TOPBAR -->
```

```
<header class="topbar">
```

```
<div>
```

```
<h1>Logistics Resource Allocation</h1>
```

```
<p>Intelligent package selection and vehicle optimization</p>
```

```
</div>
```

```
<div class="status">
```

```
<div class="status-dot"></div>
```

```
System Ready
```

```
</div>
```

```
</header>
```

```
<div class="content">
```



<!-- ===== KPI ===== -->

<div class="kpi-grid">

<div class="kpi">

<div class="kpi-icon"> 📦 </div>

<h4>Total Packages</h4>

<h2 id="totalPackages">0</h2>

</div>

<div class="kpi">

<div class="kpi-icon"> 💰 </div>

<h4>Best Delivery Value</h4>

<h2 id="bestValue">₹0</h2>

</div>

<div class="kpi">

<div class="kpi-icon"> ⚖️ </div>

<h4>Weight Used</h4>

<h2 id="weightUsed">0 kg</h2>

</div>

<div class="kpi">

<div class="kpi-icon"> 🚚 </div>

<h4>Capacity Utilization</h4>

<h2 id="utilization">0%</h2>

</div>

</div>

<!-- ===== VEHICLE ===== -->

```
<div class="card">
```

```
<div class="card-header">
```

```
<div>
```

```
<h3>Vehicle Configuration</h3>
```

```
<p>Define transportation constraints</p>
```

```
</div>
```

```
</div>
```

```
<div class="control-grid">
```

```
<div>
```

```
<label>Vehicle Weight Capacity</label>
```

```
<select id="capacity">
```

```
<option value="100">100 kg</option>
```

```
<option value="500">500 kg</option>
```

<option value="1000">1,000 kg</option>

<option value="2500">2,500 kg</option>

<option value="5000">5,000 kg</option>

</select>

</div>

<div>

<label>Operational Condition</label>

<select id="demand">

<option value="normal">

Normal Demand

</option>

<option value="high">

High Priority Demand

</option>

<option value="constrained">

Capacity Constrained

</option>

</select>

</div>

</div>

</div>

<!-- ===== PACKAGE INPUT ===== -->

<div class="card">

<div class="card-header">

<div>

<h3>Package Management</h3>

<p>Add packages to the optimization dataset</p>

</div>

</div>

```
<div class="control-grid">
```

```
<div>
```

```
<label>Package Weight (kg)</label>
```

```
<input
 type="number"
 id="packageWeight"
 placeholder="Example: 25"
 min="1">
```

```
</div>
```

```
<div>
```

```
<label>Delivery Value (₹)</label>
```

```
<input
 type="number"
 id="packageValue"
 placeholder="Example: 500"
 min="1">
```

```
</div>
```

```
<div>
```

```
<label>Priority</label>
```

```
<select id="packagePriority">
```

```
<option value="1">Low Priority</option>
```

```
<option value="2">Medium Priority</option>
```

```
<option value="3">High Priority</option>
```

```
</select>
```

```
</div>
```

```
</div>
```

```
<div class="button-row">
```

```
<button class="primary" onclick="addPackage()">
```

```
+ Add Package
```

```
</button>
```

<button class="green" onclick="loadSampleData()">

Load Sample Dataset

</button>

<button class="red" onclick="clearPackages()">

Clear Dataset

</button>

</div>

<p id="message"

style="margin-top:12px;color:#2563eb;font-size:13px;">

</p>

</div>

<!-- ===== ALGORITHMS ===== -->

<div class="card">

<div class="card-header">

<div>

<h3>Optimization Algorithms</h3>



<p>Select an algorithm to allocate packages</p>

</div>

</div>

<div class="algorithm-grid">

<div class="algorithm"  
  onclick="runDP()">

<div class="algorithm-icon"> 🧠 </div>

<strong>Dynamic Programming</strong>

<small>

  Optimal solution using 0/1 Knapsack

</small>

</div>

<div class="algorithm"  
  onclick="runGreedy()">

<div class="algorithm-icon"> ⚡ </div>

<strong>Greedy Algorithm</strong>

<small>

Fast selection using value efficiency

</small>

</div>

<div class="algorithm"

onclick="runHybrid()">

<div class="algorithm-icon"> 🚀 </div>

<strong>Hybrid Algorithm</strong>

<small>

Greedy selection with local improvement

</small>

</div>

</div>

```
<div class="button-row">
```

```
 <button class="primary"
 onclick="runAll()">
```

Compare All Algorithms

```
</button>
```

```
</div>
```

```
</div>
```

```
<!-- ===== RESULT ===== -->
```

```
<div class="card">
```

```
 <div class="card-header">
```

```
 <div>
```

```
 <h3>Optimization Result</h3>
```

```
 <p id="algorithmName">
```

Run an algorithm to view results

</p>

</div>

</div>

<div class="result-grid">

<div class="result">

<span>Algorithm</span>

<strong id="resultAlgorithm">-</strong>

</div>

<div class="result">

<span>Packages Selected</span>

<strong id="resultPackages">0</strong>

</div>

<div class="result">

<span>Total Weight</span>

<strong id="resultWeight">0 kg</strong>

</div>

<div class="result">

<span>Total Value</span>

<strong id="resultValue">₹0</strong>

</div>

<div class="result">

<span>Execution Time</span>

<strong id="resultTime">0 ms</strong>

</div>

</div>

<div class="progress-section">

<div class="progress-label">

<span>Vehicle Capacity Utilization</span>

<span id="progressText">0%</span>

</div>

<div class="progress">

<div

class="progress-bar"

id="progressBar">

</div>

</div>

</div>

</div>

<!-- ===== SELECTED PACKAGES ===== -->

```
<div class="card">
```

```
<div class="card-header">
```

```
<div>
```

```
<h3>Selected Packages</h3>
```

```
<p>Packages chosen for transportation</p>
```

```
</div>
```

```
</div>
```

```
<div class="table-container">
```

```
<table>
```

```
<thead>
```

```
<tr>
```

```
<th>Package</th>
```

```
<th>Weight</th>
```

<th>Value</th>

<th>Priority</th>

</tr>

</thead>

<tbody id="selectedTable">

</tbody>

</table>

</div>

</div>

<!-- ===== PACKAGE DATA ===== -->

<div class="card">

<div class="card-header">

<div>



<h3>Package Dataset</h3>

<p>Available packages</p>

</div>

</div>

<div class="table-container">

<table>

<thead>

<tr>

<th>Package</th>

<th>Weight</th>

<th>Value</th>

<th>Priority</th>

</tr>

</thead>

<tbody id="packageTable">

</tbody>

</table>

</div>

</div>

<!-- ===== COMPARISON ===== -->

<div class="card">

<div class="card-header">

<div>

<h3>Algorithm Performance Comparison</h3>

<p>Compare solution quality and computational performance</p>

</div>

</div>

<div class="comparison-grid" id="comparisonCards">

<div class="compare-card">

<h4>Dynamic Programming</h4>

<div class="metric">

<div class="metric-header">

<span>Value</span>

<span id="dpValue">₹0</span>

</div>

<div class="progress">

<div

class="progress-bar"

id="dpBar">

</div>

</div>

</div>

<div class="metric">

<div class="metric-header">

<span>Utilization</span>

<span id="dpUtil">0%</span>

</div>

</div>

</div>

<div class="compare-card">

<h4>Greedy</h4>

<div class="metric">

<div class="metric-header">

<span>Value</span>

<span id="greedyValue">₹0</span>

</div>

<div class="progress">

<div  
class="progress-bar"  
id="greedyBar">  
</div>

</div>

</div>

<div class="metric">

<div class="metric-header">

<span>Utilization</span>

<span id="greedyUtil">0%</span>

</div>

</div>

</div>

<div class="compare-card">

<h4>Hybrid</h4>

<div class="metric">

<div class="metric-header">

<span>Value</span>

<span id="hybridValue">₹0</span>

</div>

<div class="progress">

<div

class="progress-bar"

id="hybridBar">

</div>

</div>

</div>

<div class="metric">

<div class="metric-header">

<span>Utilization</span>

<span id="hybridUtil">0%</span>

</div>

</div>

</div>

</div>

<div class="table-container"  
style="margin-top:25px;">

<table>

<thead>

<tr>

<th>Algorithm</th>

<th>Packages</th>

<th>Weight</th>

<th>Total Value</th>

<th>Utilization</th>

<th>Time</th>

</tr>

</thead>

<tbody id="comparisonTable">

</tbody>

</table>

</div>

</div>



<footer>

Logistics Resource Allocation System

• Dynamic Programming • Greedy • Hybrid

</footer>

</div>

</main>

</div>

<script>

/\* =====

GLOBAL DATA

===== \*/

let packages = [];

let packageNumber = 1;

/\* =====

## ADD PACKAGE

```
===== */
```

```
function addPackage() {

 let weight =
 Number(document.getElementById("packageWeight").value);

 let value =
 Number(document.getElementById("packageValue").value);

 let priority =
 Number(document.getElementById("packagePriority").value);

 if (weight <= 0 || value <= 0) {

 showMessage(
 "Please enter valid package details."
);

 return;
 }

 packages.push({
```

```
id: "P" + packageNumber,
```

```
weight: weight,
```

```
value: value,
```

```
priority: priority
```

```
});
```

```
packageNumber++;
```

```
document.getElementById("packageWeight").value = "";
```

```
document.getElementById("packageValue").value = "";
```

```
displayPackages();
```

```
showMessage("Package added successfully.");
```

```
}
```

```
/* =====
SAMPLE DATA
===== */
```

```
function loadSampleData() {

 packages = [

 {id:"P1", weight:20, value:100, priority:2},

 {id:"P2", weight:40, value:200, priority:3},

 {id:"P3", weight:30, value:150, priority:1},

 {id:"P4", weight:50, value:300, priority:3},

 {id:"P5", weight:10, value:80, priority:2},

 {id:"P6", weight:60, value:250, priority:1},

 {id:"P7", weight:25, value:180, priority:3},

 {id:"P8", weight:35, value:220, priority:2},

 {id:"P9", weight:15, value:120, priority:3},

 {id:"P10", weight:45, value:270, priority:1}
```

```
];
```

```
packageNumber = 11;
```

```
displayPackages();
```

```
showMessage(
 "Sample dataset loaded successfully."
);
```

```
}
```

```
/* =====

 DISPLAY PACKAGES

===== */
```

```
function displayPackages() {
```

```
 let table =
 document.getElementById("packageTable");
```

```
table.innerHTML = "";
```

```
packages.forEach(p => {
```

```
 let priorityText =
```

```
 p.priority === 3
```

```
 ? "High"
```

```
 : p.priority === 2
```

```
 ? "Medium"
```

```
 : "Low";
```

```
 let priorityClass =
```

```
 p.priority === 3
```

```
 ? "high"
```

```
 : p.priority === 2
```

```
 ? "medium"
```

```
 : "low";
```

```
 table.innerHTML += `
```

```
<tr>
```

```
 <td>${p.id}</td>
```

```
<td>${p.weight} kg</td>
```

```
<td>₹${p.value}</td>
```

```
<td>
```

```

```

```
 ${priorityText}
```

```

```

```
</td>
```

```
</tr>
```

```
`;
```

```
});
```

```
document.getElementById("totalPackages")
```

```
 .innerText = packages.length;
```

```
}
```

```
/* =====
```

```
CLEAR
```

```
===== */
```

```
function clearPackages() {

 packages = [];

 packageNumber = 1;

 displayPackages();

 document.getElementById("selectedTable")
 .innerHTML = "";

 document.getElementById("comparisonTable")
 .innerHTML = "";

 document.getElementById("bestValue")
 .innerText = "₹0";

 document.getElementById("weightUsed")
 .innerText = "0 kg";

 document.getElementById("utilization")
```



```
.innerText = "0%";
```

```
showMessage("Dataset cleared.");
```

```
}
```

```
/* =====
```

```
MESSAGE
```

```
===== */
```

```
function showMessage(text) {
```

```
 document.getElementById("message")
```

```
 .innerText = text;
```

```
}
```

```
/* =====
```

```
CAPACITY
```

```
===== */
```

```
function getCapacity() {
```

```
 let capacity =
```

```
Number(document.getElementById("capacity").value);
```

```
let demand =
```

```
document.getElementById("demand").value;
```

```
if (demand === "constrained") {
```

```
 capacity = capacity / 2;
```

```
}
```

```
return capacity;
```

```
}
```

```
/* =====
```

```
DYNAMIC PROGRAMMING
```

```
===== */
```

```
function dynamicProgramming() {
```

```
 let capacity = getCapacity();
```

```
let start = performance.now();
```

```
let dp = Array.from(

 {
 length: packages.length + 1
 },
```

```
 () =>
 Array(capacity + 1).fill(0)
```

```
);
```

```
for (
 let i = 1;
 i <= packages.length;
 i++)
) {
```

```
 let p = packages[i - 1];
```

```
 for (
 let w = 0;
```

```

 w <= capacity;
 w++;
) {

 if (p.weight <= w) {

 dp[i][w] = Math.max(

 dp[i - 1][w],

 p.value +
 dp[
 i - 1
][
 w - p.weight
]

);

 }

 else {

 dp[i][w] =
 dp[i - 1][w];

 }
 }

```

```
}
```

```
}
```

```
let selected = [];
```

```
let w = capacity;
```

```
for (
```

```
 let i = packages.length;
```

```
 i > 0;
```

```
 i--
```

```
) {
```

```
 if (
```

```
 dp[i][w] !==
```

```
 dp[i - 1][w]
```

```
) {
```

```
 selected.push(
```

```
 packages[i - 1]
```

```
);
```

```
 w -=
```

```
packages[i - 1].weight;
```

```
}
```

```
}
```

```
let end = performance.now();
```

```
return createResult(
```

```
"Dynamic Programming",
```

```
selected,
```

```
end - start
```

```
);
```

```
}
```

```
/* =====
```

```
GREEDY
```

```
===== */
```

```
function greedy() {

 let capacity = getCapacity();

 let start = performance.now();

 let sorted = [...packages];

 sorted.sort((a, b) => {

 let scoreA =
 (a.value * a.priority)
 / a.weight;

 let scoreB =
 (b.value * b.priority)
 / b.weight;

 return scoreB - scoreA;

 });
```

```
let selected = [];
```

```
let weight = 0;
```

```
for (let p of sorted) {
```

```
 if (
```

```
 weight + p.weight
```

```
 <= capacity
```

```
) {
```

```
 selected.push(p);
```

```
 weight += p.weight;
```

```
 }
```

```
}
```

```
let end = performance.now();
```

```
return createResult(
```

```
 "Greedy",
```



```

 selected,
 end - start
);
}
/* =====
HYBRID
===== */

function hybrid() {
 let capacity = getCapacity();
 let start = performance.now();
 let sorted = [...packages];
 sorted.sort((a, b) => {
 let scoreA =
 (a.value * a.priority)
 / (a.weight + 0.5);
 let scoreB =
 (b.value * b.priority)
 / (b.weight + 0.5);
 return scoreB - scoreA;

 });
 let selected = [];
 let weight = 0;
 /* Initial Greedy Selection */

 for (let p of sorted) {

```

```

 if (
 weight + p.weight
 <= capacity
) {

 selected.push(p);

 weight += p.weight;
 }
}

/* Local Improvement */

for (let p of packages) {

 if (
 selected.includes(p)
)
 continue

 for (
 let i = 0;
 i < selected.length;
 i++
) {
 let currentValue =
 selected.reduce(
 (sum, x) =>
 sum + x.value,

```

```

 0
);
 let newWeight =
 weight -
 selected[i].weight +
 p.weight;
 let newValue =
 currentValue -
 selected[i].value +
 p.value;
 if (
 newWeight <= capacity &&
 newValue > currentValue
) {
 weight = newWeight;
 selected[i] = p;
 break;
 }
}
let end = performance.now();
return createResult(

 "Hybrid",

 selected,

```

end - start

);

}

/\* =====

CREATE RESULT

===== \*/

function createResult(

name,

selected,

time

) {

let weight =

selected.reduce(

(sum, p) =>

sum + p.weight,

0

);

let value =

selected.reduce(

(sum, p) =>

sum + p.value,

0

);

```

let capacity =
 getCapacity();
let utilization =
 (weight / capacity) * 100;
return {
 name,
 selected,
 weight,
 value,
 utilization,
 time
};
}
/* =====
SHOW RESULT
===== */

```

```

function showResult(result) {

 document.getElementById(
 "resultAlgorithm"
).innerText = result.name;
 document.getElementById(
 "algorithmName"
).innerText =
 result.name +
 " optimization completed";
}

```

```
document.getElementById(
 "resultPackages"
).innerText =
 result.selected.length;
document.getElementById(
 "resultWeight"
).innerText =
 result.weight + " kg";
document.getElementById(
 "resultValue"
).innerText =
 "₹" + result.value;
document.getElementById(
 "resultTime"
).innerText =
 result.time.toFixed(4) +
 " ms";
document.getElementById(
 "utilization"
).innerText =
 result.utilization.toFixed(1)
 + "%";
document.getElementById(
 "bestValue"
).innerText =
 "₹" + result.value;
```

```
document.getElementById(
 "weightUsed"
).innerText =
 result.weight + " kg";
document.getElementById(
 "progressText"
).innerText =
 result.utilization.toFixed(1)
 + "%";
document.getElementById(
 "progressBar"
).style.width =
 Math.min(
 result.utilization,
 100
) + "%";
let table =
 document.getElementById(
 "selectedTable"
);
table.innerHTML = "";
result.selected.forEach(p => {
 let priorityText =
 p.priority === 3
 ? "High"
 : p.priority === 2
```

```

 ? "Medium"
 : "Low";
let priorityClass =
 p.priority === 3
 ? "high"
 : p.priority === 2
 ? "medium"
 : "low";
table.innerHTML += `
<tr>
 <td>
 ${p.id}
 </td>
 <td>${p.weight} kg</td>
 <td>₹${p.value}</td>
 <td>

 ${priorityText}

 </td>
</tr>
 });
}
/* =====
RUN DP
===== */

```



```

function runDP() {
 if (packages.length === 0) {
 alert(
 "Please load or add packages first."
);
 return;
 }
 showResult(
 dynamicProgramming()
);
}
/* =====

RUN GREEDY

===== */

function runGreedy() {

 if (packages.length === 0) {
 alert(
 "Please load or add packages first."
);
 return;
 }
 showResult(
 greedy()
);
}
/* =====

```

## RUN HYBRID

```
===== */

function runHybrid() {
 if (packages.length === 0) {
 alert(
 "Please load or add packages first."
);
 return;
 }
 showResult(
 hybrid()
);
}
/* =====
```

## COMPARE ALL

```
===== */

function runAll() {
 if (packages.length === 0) {
 alert(
 "Please load or add packages first."
);
 return;
 }
 let dp =
 dynamicProgramming();
 let gr =
 greedy();
```

```
let hy =
 hybrid();
let results = [
 dp,
 gr,
 hy
];

let maxVal =
 Math.max(
 dp.value,
 gr.value,
 hy.value
);

/* Comparison Cards */
document.getElementById(
 "dpValue"
).innerText =
 "₹" + dp.value;

document.getElementById(
 "greedyValue"
).innerText =
 "₹" + gr.value;

document.getElementById(
 "hybridValue"
).innerText =
 "₹" + hy.value;

document.getElementById(
 "hybridValue"
```

```
 "dpBar"
).style.width =
 (dp.value / maxValue * 100)
 + "%";
document.getElementById(
 "greedyBar"
).style.width =
 (gr.value / maxValue * 100)
 + "%";
document.getElementById(
 "hybridBar"
).style.width =
 (hy.value / maxValue * 100)
 + "%";
document.getElementById(
 "dpUtil"
).innerText =
 dp.utilization.toFixed(1)
 + "%";
document.getElementById(
 "greedyUtil"
).innerText =
 gr.utilization.toFixed(1)
 + "%";
document.getElementById(
 "hybridUtil"
).innerText =
```

```

 hy.utilization.toFixed(1)
 + "%";
/* Comparison Table */
let table =
 document.getElementById(
 "comparisonTable"
);
table.innerHTML = "";
results.forEach(r => {
 table.innerHTML += `

<tr>
 <td>
 ${r.name}
 </td>
 <td>
 ${r.selected.length}
 </td>
 <td>
 ${r.weight} kg
 </td>
 <td>
 ₹${r.value}
 </td>
 <td>
 ${r.utilization.toFixed(2)}%
 </td>

```

```
<td>

 ${r.time.toFixed(4)} ms

</td>

</tr>

});

/* Show Hybrid as final result */

showResult(hy);

}

/* =====

INITIAL SAMPLE DATA

===== */

loadSampleData();

</script>

</body>

</html>

'''
```